Contrato de Servicio: HPC Service + Scheduler

1. Información general

Campo	Valor
Nombre del servicio	HPC Service (incluye Scheduler como sub-módulo interno)
Descripción	Servicio central de negocio de UPB-CIENTÍFICA: recibe trabajos científicos, los valida, los planifica y los ejecuta en paralelo sobre el Cluster MPI, devolviendo resultados al usuario.
Objetivo	Permitir que un investigador envíe código y datasets, y obtenga resultados ejecutados en paralelo, sin necesidad de administrar el clúster directamente.
Responsabilidad	Recepción y validación de trabajos, planificación (Scheduler), asignación de nodos, comunicación con MPI, consulta de estado, gestión de resultados.
Tecnología	Java + RMI (impuesta)
Protocolo de comunicación	RMI (interfaces remotas Java)
Servicios consumidores	Clientes (Web, Windows, Linux, Android, iOS).
Servicios proveedores	Authentication Service (`ValidarToken`), Distributed Storage Service (lectura de código/dataset, escritura de resultados), Cluster MPI (ejecución real, componente interno del propio servicio, no un servicio SOA externo).

2. Funcionalidades (operaciones expuestas vía interfaz RMI)

1. EnviarTrabajo
2. ValidarTrabajo (interno, invocado dentro de EnviarTrabajo)
3. ConsultarEstadoTrabajo
4. ObtenerResultado
5. CancelarTrabajo
6. ListarTrabajosUsuario
7. ObtenerLogsTrabajo
8. DefinirPrioridadTrabajo
9. ReintentarTrabajo

Operaciones internas del Scheduler (no expuestas directamente a clientes, sino consumidas por el propio HPC Service):

10. EncolarTrabajo
11. AsignarNodos
12. ObtenerEstadoCola
13. ObtenerEstadoCluster
14. NotificarFinalizacionJob (callback desde MPI hacia el Scheduler)
15. LiberarNodos

---

3. Contrato de cada operación

3.1 EnviarTrabajo

Nombre: `enviarTrabajo` **Descripción:** Recibe un trabajo científico (referencia a código y dataset ya almacenados, o los recibe directamente) y lo entrega al Scheduler para su planificación. **Protocolo:** RMI — método remoto de la interfaz `HPCServiceRemote`

Parámetros de entrada:

Nombre	Tipo	Obligatorio	Descripción	Restricciones/Validaciones
`token`	String	Sí	Token de sesión válido	Validado contra Authentication Service
`nombreTrabajo`	String	Sí	Nombre descriptivo del trabajo	1-128 caracteres
`rutaCodigo`	String	Sí	Ruta en Storage del código MPI a ejecutar	Debe existir y el usuario debe tener permiso de lectura
`rutaDataset`	String	No	Ruta en Storage del dataset de entrada	Si se omite, el trabajo no requiere dataset externo
`numeroProcesos`	int	Sí	Cantidad de procesos MPI solicitados	Mínimo 1, máximo definido por política de cuota de cómputo del usuario/rol
`tiempoMaximoEjecucion`	int (segundos)	Sí	Límite de ejecución antes de terminación forzada	Máximo permitido según política institucional
`prioridad`	enum	No	`BAJA`, `NORMAL`, `ALTA`	Por defecto `NORMAL`; `ALTA` requiere permiso `hpc:prioridad-alta`
`parametrosEjecucion`	Map<String,String>	No	Argumentos adicionales pasados al programa MPI	—

Datos de salida:

Nombre	Tipo	Descripción
`trabajoId`	UUID	Identificador único del trabajo
`estadoInicial`	enum	Siempre `PENDIENTE` al momento de la respuesta
`posicionEstimadaCola`	int	Posición aproximada en la cola al momento del envío (informativo, puede cambiar)
`fechaEnvio`	datetime	—

Posibles errores (excepciones remotas RMI):

Código	Descripción	Posible solución
`HPC-001`	Código no encontrado o sin permiso de lectura	Verificar `rutaCodigo` y permisos en Storage
`HPC-002`	Dataset no encontrado	Verificar `rutaDataset`
`HPC-003`	Número de procesos excede la cuota de cómputo del usuario	Reducir `numeroProcesos` o solicitar ampliación
`HPC-004`	Formato de código no soportado (no es un binario/script MPI válido)	Verificar formato esperado por el clúster
`HPC-005`	Tiempo máximo de ejecución excede el límite institucional	Ajustar `tiempoMaximoEjecucion`
`HPC-006`	Token inválido/expirado	Reautenticar
`HPC-007`	Prioridad `ALTA` solicitada sin permiso	Solicitar permiso o usar `NORMAL`

Reglas de negocio:

- Un trabajo nunca se envía directamente a MPI: **siempre** pasa primero por validación (código/dataset accesibles, cuota de cómputo no excedida) y luego por el Scheduler.
- La cuota de cómputo (procesos simultáneos, tiempo total mensual) es independiente de la cuota de almacenamiento de Storage — son dos límites distintos que deben evaluarse por separado.

- `tiempoMaximoEjecucion` es una garantía para el clúster, no una sugerencia: superado ese tiempo, el trabajo se termina forzosamente (ver `NotificarFinalizacionJob`, estado `FALLIDO_TIMEOUT`).

Seguridad: Requiere token válido y permiso `hpc:enviar-trabajo`. `prioridad=ALTA` requiere permiso adicional `hpc:prioridad-alta`.

Dependencias: Authentication Service (token/permisos), Distributed Storage Service (verificar existencia y permisos de código/dataset), Scheduler (interno).

Flujo interno:

1. Presentación (RMI) recibe la invocación remota.
2. Aplicación valida token contra Authentication Service.
3. Dominio valida existencia y permisos de código/dataset contra Storage.
4. Dominio valida cuota de cómputo del usuario.
5. Se crea la entidad `TrabajoHPC` en estado `PENDIENTE`.
6. Se invoca `EncolarTrabajo` en la capa de Scheduling.
7. Se responde al cliente con `trabajoId` (la ejecución real es asíncrona a partir de aquí).

3.2 ConsultarEstadoTrabajo

Entrada: `token`, `trabajoId`. **Salida:** `estado` (`PENDIENTE`, `EN_COLA`, `EJECUTANDO`, `COMPLETADO`, `FALLIDO`, `FALLIDO_TIMEOUT`, `CANCELADO`), `progreso` (float 0-1, si el clúster reporta avance), `nodosAsignados` (array, si ya está `EJECUTANDO`), `mensajeError` (si aplica). **Errores:** `HPC-101` (trabajo no encontrado), `HPC-102` (trabajo no pertenece al usuario y no tiene permiso de consulta administrativa). **Regla de negocio:** el estado se consulta contra el registro del Scheduler, que es la única fuente de verdad — el HPC Service nunca "adivina" el estado por tiempo transcurrido.

3.3 ObtenerResultado

Entrada: `token`, `trabajoId`. **Salida:** `rutaResultado` (ubicación en Storage donde ya se almacenó el resultado), `metadatosResultado` (tamaño, fecha, checksum — delegado de Storage). **Errores:** `HPC-103` (el trabajo aún no ha completado — se debe consultar `ConsultarEstadoTrabajo` primero), `HPC-104` (el trabajo falló y no produjo resultado válido). **Regla de negocio:** esta operación **no transporta el contenido del resultado por RMI** — solo devuelve la referencia; el cliente descarga el contenido real desde Storage (evita saturar el canal RMI con transferencias grandes de datos).

3.4 CancelarTrabajo

Entrada: `token`, `trabajoId`. **Salida:** confirmación, `estadoFinal` (`CANCELADO`). **Errores:** `HPC-101` (no encontrado), `HPC-105` (el trabajo ya completó/falló, no se puede cancelar

retroactivamente). **Reglas de negocio:** si el trabajo ya está `EJECUTANDO`, cancelar implica terminar los procesos MPI activos y liberar los nodos asignados de inmediato (no esperar a que terminen naturalmente).

3.5 ListarTrabajosUsuario / ObtenerLogsTrabajo

Entrada (ListarTrabajosUsuario): `token`, `filtroEstado` (opcional), `paginacion`. **Salida:** lista de `TrabajoHPC` (resumen).

Entrada (ObtenerLogsTrabajo): `token`, `trabajoId`. **Salida:** `logsEjecucion` (texto o referencia a un archivo de log en Storage si es muy extenso). **Error:** `HPC-106` (logs no disponibles aún, trabajo no ha iniciado ejecución).

3.6 DefinirPrioridadTrabajo / ReintentarTrabajo

Entrada (DefinirPrioridadTrabajo): `token`, `trabajoId`, `nuevaPrioridad`. Solo aplica si el trabajo sigue `EN_COLA` (no si ya está `EJECUTANDO`). **Entrada (ReintentarTrabajo):** `token`, `trabajoId` (debe estar en estado `FALLIDO` o `FALLIDO_TIMEOUT`). **Salida:** `nuevoTrabajoId` (se crea un trabajo nuevo enlazado al original, nunca se reintenta "in place" para preservar el historial de la ejecución fallida). **Error:** `HPC-107` (no se puede reintentar un trabajo que no falló).

3.7 EncolarTrabajo (Scheduler, interno)

Descripción: Recibe un `TrabajoHPC` validado y lo coloca en la cola de planificación según su prioridad. **Entrada:** `TrabajoHPC` completo. **Salida:** confirmación de encolado + `posicionCola`. **Regla de negocio:** la cola es de **prioridad con envejecimiento** (aging): un trabajo `NORMAL` que espera demasiado tiempo incrementa su prioridad efectiva para evitar inanición frente a un flujo constante de trabajos `ALTA`.

3.8 AsignarNodos (Scheduler, interno)

Descripción: Cuando hay recursos disponibles, selecciona qué nodos del Cluster MPI ejecutarán el siguiente trabajo de la cola. **Entrada:** `trabajoId`, `numeroProcesos` requeridos. **Salida:** `listaNodosAsignados`. **Error:** `HPC-201` (no hay suficientes nodos disponibles actualmente — el trabajo permanece en cola, no es un error fatal, es un estado transitorio). **Regla de negocio:** el algoritmo de asignación debe considerar carga actual de cada nodo, no solo disponibilidad binaria — evita concentrar todos los trabajos en los primeros nodos "libres" ignorando el balanceo.

3.9 ObtenerEstadoCola / ObtenerEstadoCluster

Entrada: ninguna (o `token` administrativo para detalle extendido). **Salida (ObtenerEstadoCola):** `trabajosEnCola`, `tiempoEsperaPromedioEstimado`. **Salida**

(ObtenerEstadoCluster): `nodosTotales`, `nodosDisponibles`, `nodosEnFalla`, `capacidadUsoActual`. **Consumidor principal:** Monitoring Service.

3.10 NotificarFinalizacionJob (callback desde MPI, interno)

Descripción: El Cluster MPI notifica al Scheduler cuando un trabajo termina (exitosamente, con error, o por timeout). **Entrada:** `trabajoId`, `estadoFinal`, `codigoSalida`, `rutaResultadoTemporal` (antes de moverse a Storage definitivo). **Salida:** confirmación interna. **Reglas de negocio:**

- Al recibir esta notificación, el Scheduler debe: (1) mover el resultado a su ubicación definitiva en Storage, (2) liberar los nodos asignados (`LiberarNodos`), (3) actualizar el estado del `TrabajoHPC`, (4) emitir el evento `TrabajoFinalizado`.
- Si el resultado no puede moverse a Storage (ej. falla de red en ese instante), el trabajo se marca como `COMPLETADO_PENDIENTE_ALMACENAMIENTO` y se reintenta la operación de guardado — nunca se pierde un resultado ya calculado por un problema de infraestructura de almacenamiento.

---

4. Modelo de datos

TrabajoHPC

Atributo	Tipo	Descripción	Restricciones
`id`	UUID	Identificador	PK
`usuarioId`	UUID	Propietario	FK lógica a Authentication
`nombreTrabajo`	string	—	—
`rutaCodigo` / `rutaDataset`	string	Referencias a Storage	—
`numeroProcesos`	int	—	⩾ 1
`prioridad`	enum	`BAJA`, `NORMAL`, `ALTA`	—
`estado`	enum	`PENDIENTE`, `EN_COLA`, `EJECUTANDO`, `COMPLETADO`, `COMPLETADO_PENDIENTE_ALMACENAMIENTO`, `FALLIDO`, `FALLIDO_TIMEOUT`, `CANCELADO`	—
`fechaEnvio` / `fechaInicio` / `fechaFin`	datetime	—	—
`trabajoOriginalId`	UUID (nullable)	Si es un reintento, referencia al trabajo original	—

NodoCluster

Atributo	Tipo	Descripción
`id`	string	Identificador físico del nodo
`estado`	enum	`DISPONIBLE`, `OCUPADO`, `EN_FALLA`
`capacidadProcesos`	int	Máximo de procesos MPI que puede alojar
`cargaActual`	int	Procesos actualmente en ejecución

AsignacionRecursos

Atributo	Tipo	Descripción
`trabajoId`	UUID	—
`nodosAsignados`	array<string>	IDs de nodo
`fechaAsignacion`	datetime	—

Resultado

Atributo	Tipo	Descripción
`trabajoId`	UUID	—
`rutaStorage`	string	Ubicación definitiva
`codigoSalida`	int	—
`tamanoBytes`	long	—

5. Eventos que genera el servicio

Evento	Disparado cuando	Consumidores típicos
`TrabajoEncolado`	`EnviarTrabajo` exitoso	Monitoring Service
`TrabajoIniciado`	`AsignarNodos` exitoso y MPI comienza ejecución	Monitoring Service
`TrabajoFinalizado`	`NotificarFinalizacionJob` procesado (éxito o fallo)	Cliente (notificación), Monitoring Service
`NodoEnFalla`	Un nodo deja de responder durante ejecución	Monitoring Service (alerta crítica), Scheduler (reasignación)
`ColaSaturada`	La cola supera un umbral configurable de trabajos pendientes	Monitoring Service (alerta de capacidad)

6. Integración

- **Quién consume:** todos los clientes finales (vía RMI).

- **Quién responde:** HPC Service, que internamente orquesta al Scheduler y este a su vez al Cluster MPI (componente interno, no un servicio SOA independiente).
- **Protocolo:** RMI para todo lo expuesto a clientes; la comunicación Scheduler↔MPI es interna del servicio (puede ser RMI también, o un mecanismo más directo, a decisión del equipo, pero no se expone como contrato externo).
- **Momento del flujo:** posterior a Authentication (identidad) y a Storage (validación de código/dataset); es el flujo de negocio central de toda la plataforma.

---

7. Consideraciones de escalabilidad

- **Qué puede crecer:** el número de trabajos concurrentes en cola es lo que más presiona al Scheduler; `ConsultarEstadoTrabajo` puede volverse una operación de alta frecuencia si los clientes hacen polling agresivo.
- **Cuellos de botella posibles:** un Scheduler centralizado es, por diseño, un punto de contención — mitigarlo con una cola eficiente (no recorrer linealmente todos los trabajos pendientes para decidir el siguiente) es más importante que intentar paralelizar el propio Scheduler en esta etapa del proyecto.
- **Mejoras futuras:** notificación push de cambio de estado (en vez de que el cliente haga polling constante a `ConsultarEstadoTrabajo`); particionamiento del clúster por tipo de carga de trabajo.

---

8. Casos límite

Operación	Caso límite	Comportamiento esperado
`EnviarTrabajo`	El código referenciado se elimina de Storage justo después de validarlo pero antes de ejecutarlo	El Scheduler debe re-validar la existencia del código al momento de asignar nodos, no confiar solo en la validación inicial
`CancelarTrabajo`	Se cancela un trabajo justo cuando `NotificarFinalizacionJob` ya está en camino desde MPI	Debe primar el resultado de finalización si ya se generó output válido — no se descarta un resultado ya calculado por una cancelación que llegó tarde
`AsignarNodos`	Un nodo asignado falla justo después de la asignación pero antes de iniciar ejecución	El Scheduler debe reintentar la asignación con otro nodo automáticamente, sin devolver error al usuario si hay recursos alternativos disponibles
`NotificarFinalizacionJob`	El trabajo excede `tiempoMaximoEjecucion` pero produce output parcial válido	Se marca `FALLIDO_TIMEOUT`, y el output parcial se conserva en Storage con una bandera de "resultado incompleto", nunca se descarta silenciosamente
`ReintentarTrabajo`	Se reintenta un trabajo cuyo código fue modificado después del fallo original	El reintento usa la versión **actual** del código (no la versión histórica), y esto debe quedar explícito en la respuesta para que el usuario no asuma que se repitió exactamente la misma ejecución

Contrato listo para implementación independiente de HPC Service y su Scheduler interno. Siguiente en la secuencia: Monitoring Service.